
Anomaly Detection in Wind Turbines

A comparative study using SCADA time-series data and deep learning

DATASET

CARE_To_Compare · Wind Farm C

MODELS

CNN · LSTM · CNN-LSTM · Autoencoders

AUTHOR & DATE

Thiago Heleno · April 2026

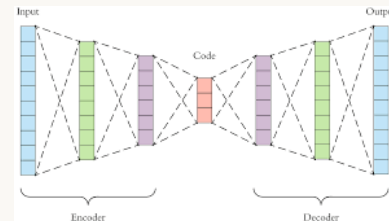
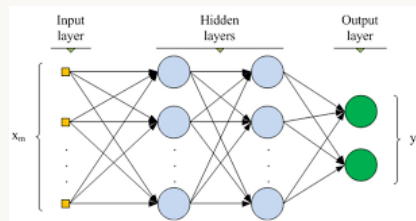
The Research Question

Can deep learning automatically detect mechanical anomalies in wind turbines from raw SCADA sensor readings — and which paradigm performs better under severe class imbalance?

SUPERVISED - Based on a Chinese paper

Notebook 1 • CNN-LSTM

The model learns the difference between Normal and Anomaly from labeled examples. Requires annotated fault data. Output: binary label.



SEMI-SUPERVISED - Nb3 based on the CARE Score

Notebooks 2 & 3 • Autoencoders

The model learns only what normal looks like. Anomalies are deviations — detected via reconstruction error.

Why this matters

- Wind turbines are expensive to maintain and hard to access offshore
- Early fault detection reduces downtime and repair costs significantly
- SCADA data is abundant but anomaly labels are rare and expensive
- Class imbalance (98% normal / 2% anomaly) is the central technical challenge

Core question: does semi-supervised anomaly detection outperform supervised classification when fault labels are scarce and data is heavily imbalanced?

The Dataset — CARE_To_Compare Wind Farm C

3.18M

Total rows

238

Physical sensors

58

Recorded events

10 min

Sampling interval

Sensor Columns After Preprocessing

238 physical sensors expand to 952 raw columns because angular measurements (degrees) are split into sin/cos pairs — preserving circular continuity where $359^\circ \approx 1^\circ$.

Class Imbalance — The Central Challenge

Normal 98.1%		
Class	Samples	Share
Normal	3,126,239	98.1%
Anomaly	60,897	1.9%

Event Composition

Type	NB1 / NB2	NB3 (Keras)
Anomaly events	27	51
Normal events	31	7

Imbalance ratio: 51:1. A model that always predicts "Normal" reaches 98% accuracy while detecting zero faults. Accuracy is therefore meaningless here.

Train / Validation / Test Splits

NB 1

Split by Event

Partitioned at event boundaries — no temporal overlap between splits.

Split	Events	Anomaly	Normal
Train	40	18	22
Validation	8	4	4
Test	10	5	5

After windowing + undersampling: Train 50K normal + 50K anomaly (1:1). Test: 532K normal + 9K anomaly (58:1 real ratio).

NB 2 & NB 3 — SEMI-SUPERVISED

Temporal Split — 70 / 15 / 15

Chronological partition respecting time ordering of readings.

TRAIN 70% · 2.23M rows

VAL

TEST

Anti-leakage rule: MinMaxScaler is fitted only on normal training samples — never on validation, test, or anomaly data.

Autoencoder training: only normal windows are presented during training. The model never sees a labeled anomaly.

Preprocessing Pipeline

STEPS APPLIED ACROSS ALL NOTEBOOKS

- 1 Load 58 event CSV files from Wind Farm C
- 2 DataClipper — clips extreme outliers to Q0.1% and Q99.9% per feature
- 3 DuplicateValuesToNaN — sensors frozen for 6+ consecutive readings become NaN
- 4 SimpleImputer — forward-fill → backward-fill → fill zeros
- 5 Angle encoding (NB2/3) — each angular sensor → sin + cos pair; 238 sensors → 952 columns
- 6 MinMaxScaler [0,1] — fitted exclusively on normal training data
- 7 Feature selection → 285 features (NB1) or 305 features (NB2/3)

Why freeze/outlier detection?

Industrial SCADA hardware can freeze or spike due to communication errors. These are data quality issues, not physical anomalies, and must be removed before modelling.

Why sin/cos for angles?

Raw: $359^\circ \leftrightarrow 1^\circ = 358$ apart (wrong!). sin/cos encoding: $\approx$ identical vectors ✓. Preserves circular geometry that ML models need to generalise.

Why scale only on normal data?

If anomaly values influence the scaler's range, their out-of-distribution signals are compressed. Fitting only on normal data preserves anomaly extremes after scaling.

Feature Selection — 952 → ~300 Features

NB1 — Supervised (XGBoost)

952 raw features

952

↓ Train XGBoost on 100K sampled rows, rank by feature gain. Keep top 30%.

285 final features

285

✓ Selects features most discriminative between Normal and Anomaly

✗ Requires labels — unusable without annotated fault data

NB2 & NB3 — Unsupervised Filtering

952 raw features

952

↓ Remove variance < 0.0001 (nearly constant) → -46

906 remain

906

↓ Remove correlation with operational vars < 0.1 → -129

777 remain

777

↓ Remove inter-feature correlation > 0.95 → -472

305 final features

305

Operational reference variables: power output, wind speed, sensor_144/145 (avg/max/min/std).

Three Modelling Approaches

NOTEBOOK 1**CNN / LSTM / CNN-LSTM**

Paradigm	Supervised
Framework	PyTorch
Input	36 × 285 (window)
Output	Binary label 0/1
Loss	CrossEntropy
Balancing	Undersampling 1:1
Parameters	~160K

NOTEBOOK 2**CNN-BiLSTM Autoencoder**

Paradigm	Semi-supervised
Framework	PyTorch
Input	36 × 305 (window)
Output	Reconstruct input
Loss	MAE
Balancing	N/A — normal only
Parameters	~954K

NOTEBOOK 3**Keras MLP Autoencoder**

Paradigm	Semi-supervised
Framework	TF / Keras
Input	305 (single sample)
Output	Reconstruct input
Loss	MSE
Balancing	N/A — normal only
Parameters	~150K

Fundamental difference: NB1 is told "this is what a fault looks like." NB2 and NB3 are told "this is what healthy operation looks like — flag anything else." NB3 operates pointwise (no temporal context); NB2 operates over 6-hour windows.

CNN-LSTM — Supervised Pipeline

- 1 Load 58 CSVs → split by event (no window overlap across splits)
- 2 MinMaxScaler fitted on normal train data only → normalize [0, 1]
- 3 XGBoost feature selection on 100K sample → top 30% = 285 features
- 4 Sliding windows of 36 timesteps (6 hours) — label = Anomaly if any step is anomalous
- 5 Undersampling (train only) — keep 2.3% of normal windows → balanced 1:1 training set
- 6 Train CNN, LSTM, and CNN-LSTM with CrossEntropy loss · Adam · up to 500 epochs
- 7 Sweep thresholds on validation → choose threshold maximising F1-Score
- 8 Apply optimal threshold to test set (natural 58:1 ratio) → final metrics

Window Labelling Strategy

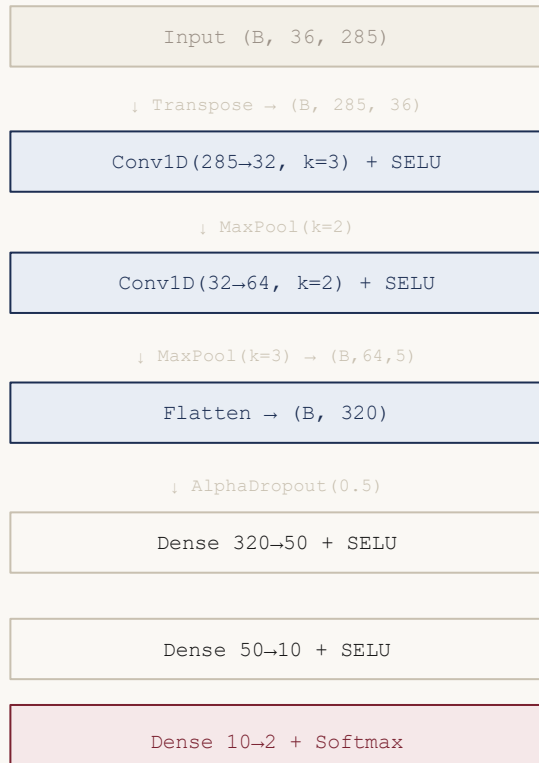
```
[ N N N N N N N N ] → 0 (Normal)
[ N N A N N N N N ] → 1 (Anomaly)
# ANY anomaly in window → label = 1
```

Training Configuration

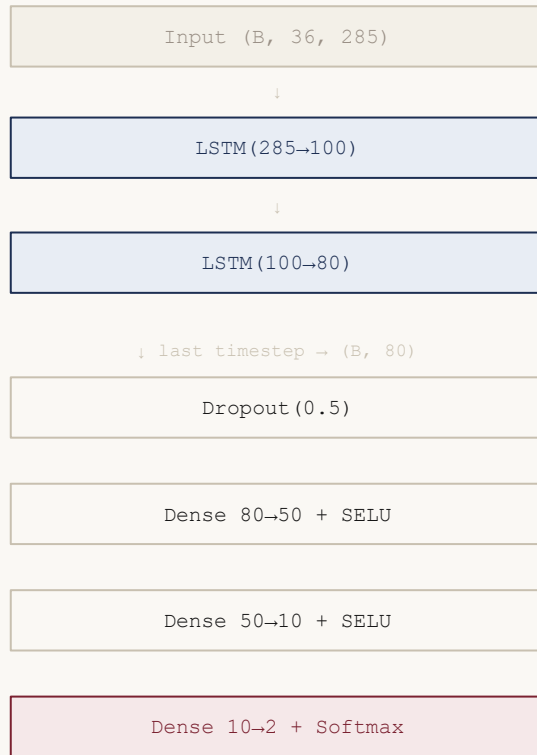
Optimizer	Adam (lr = 0.001)
Batch size	600
Max epochs	500
Early stopping	patience = 20 (val_loss)
Device	GPU (CUDA)
Loss	CrossEntropyLoss

CNN, LSTM, and CNN-LSTM Architectures

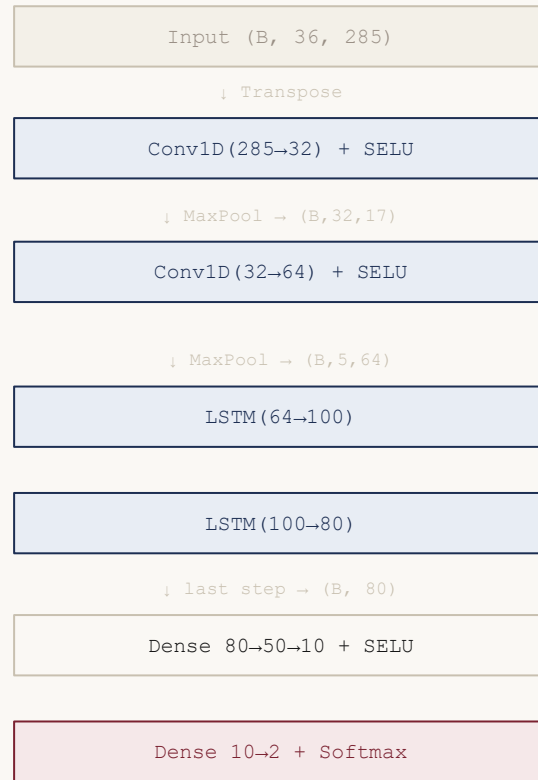
Convolutional Network



Recurrent Network



Hybrid CNN-LSTM (best)



CNN-BiLSTM Autoencoder — Semi-supervised Pipeline

- 1 Temporal split 70/15/15 — preserves chronological ordering
- 2 Preprocessing: DataClipper → DuplicateToNaN → Imputation → MinMaxScaler
- 3 Unsupervised feature selection → 305 features
- 4 Sliding windows of 36 timesteps — train and val use only normal windows
- 5 Train CNN-BiLSTM-Attention Autoencoder — MAE loss, mixed precision (AMP), 100 epochs
- 6 Compute per-feature thresholds (P95 and P99) on normal validation windows
- 7 Anomaly if ≥ 15 of 305 features exceed their individual threshold
- 8 v4 post-hoc refinements — feature weighting, temporal smoothing, informative mask

The Autoencoder Principle

Normal → Encoder → Latent → Decoder → $\approx$ Input (low reconstruction error)

Anomaly → Encoder → Latent → Decoder → $\neq$ Input (high reconstruction error → flag!)

Detection

If in the 5 windows before, there are more anomalies than normal, it flags a real anomaly in the window

CNN-BiLSTM-Attention Architecture · ~954K Parameters

ENCODER

Input: (B, 36, 305) — 6-hour window × 305 features

↓

Conv1D(305→64, k=3, pad=1) + SELU → (B, 36, 64)

↓ captures local temporal patterns

Conv1D(64→128, k=3, pad=1) + SELU → (B, 36, 128)

↓

BiLSTM(128 → 256 bidirectional) → (B, 36, 256)

↓ past + future context simultaneously

BiLSTM(256 → 128 bidirectional) → (B, 36, 128)

↓

MultiHeadAttention(embed=128, heads=4) + LayerNorm → (B, 36, 128)

DECODER

Latent: (B, 36, 128)

↓

LSTM(128→128) + SELU → (B, 36, 128)

↓

LSTM(128→128) + SELU → (B, 36, 128)

↓ Dropout(0.15)

Linear(128→256) + SELU → (B, 36, 256)

↓

Linear(256→305) → Reconstruction (B, 36, 305)

BiLSTM processes the sequence in both directions, capturing richer temporal dependencies. Multi-Head Attention learns which timesteps and features contribute most to reconstruction quality.

Keras MLP Autoencoder + Optuna

- 1 Temporal split 70/15/15 — cut points: Oct 2025 and May 2027
- 2 sklearn Pipeline: DataClipper → DuplicateToNaN → Imputation → MinMaxScaler
- 3 Unsupervised feature selection → 305 features (same strategy as NB2)
- 4 Optuna Study 1 (50 trials) — architecture hyperparameter search
- 5 Optuna Study 2 (50 trials) — adaptive threshold gamma search
- 6 Train final MLP autoencoder — MSE loss, 50 epochs, ExponentialDecay LR
- 7 Adaptive threshold: $\text{threshold}(xi) = \text{RMSE_predicted}(xi) + \text{gamma}$
- 8 Evaluate with CARE metric — per-event and per-sample scoring

Optuna Hyperparameter Search

Parameter	Range	Best
n_layers	[1, 2, 3]	2
code_size	[10, 100]	35
learning_rate	[1e-4, 1e-2]	9.795e-4
decay_rate	[0.90, 0.999]	0.9941
gamma	[0.0, 0.5]	0.4997

Adaptive Threshold

$\text{threshold}(xi) = \text{RMSE_predicted}(xi) + \text{gamma}$

Detection

“screenshots” a timestep and tries to reconstruct, if the reconstruction error is over the threshold, mark as a temporary anomaly. If the next 6 timesteps are anomaly, the event is marked as an anomaly

Dense MLP Architecture ~150K Parameters

ENCODER

Input: (B, 305)

↓

Dense(305→200) + PReLU

↓

Dense(200→100) + PReLU

↓

Bottleneck Dense(100→35) + PReLU

DECODER

Latent: (B, 35)

↓

Dense(35→100) + PReLU

↓

Dense(100→200) + PReLU

↓

Dense(200→305) linear

Pointwise vs. Windowed Processing

	NB2 (BiLSTM)	NB3 (MLP)
Temporal context	36 timesteps (6h)	None
Input shape	(B, 36, 305)	(B, 305)
Captures trends	Yes	No
Computational cost	High	Low

PReLU Activation

$\text{PReLU}(x) = \max(0, x) + \alpha \cdot \min(0, x)$ ← α is learned

Unlike standard ReLU ($\alpha=0$ fixed), the negative slope is a trained parameter allowing more flexibility in representing asymmetric sensor distributions.

Bottleneck of 35 neurons forces the encoder to compress 305 features into an essential representation of healthy operation. Anything that cannot be faithfully reconstructed is treated as anomalous.

Metrics — What We Measure and Why

CONFUSION MATRIX

	Predicted Normal	Predicted Anomaly
Actual Normal	TN True Negative	FP False Alarm
Actual Anomaly	FN Missed fault!	TP Detected!

METRIC DEFINITIONS

Metric	Formula	Best use
Precision	$TP / (TP + FP)$	% of alarms that are real faults
Recall	$TP / (TP + FN)$	% of faults successfully detected
F1-Score	$2 * P * R / (P + R)$	Harmonic balance of P and R
AUC-ROC	Area under ROC	Threshold-independent ranking
CARE Score	Composite	$CARE = (F1_2_mean + WS_mean + EF1_2 + 2.0 * Acc_mean) / 5.0$

A model predicting "Normal" always achieves 98% accuracy with zero fault detection. Accuracy is unreliable — AUC-ROC and F1 are used instead.

Sample-level vs. Event-level

Per sample — every 10-min reading evaluated. Strict: model must be correct at each timestep.

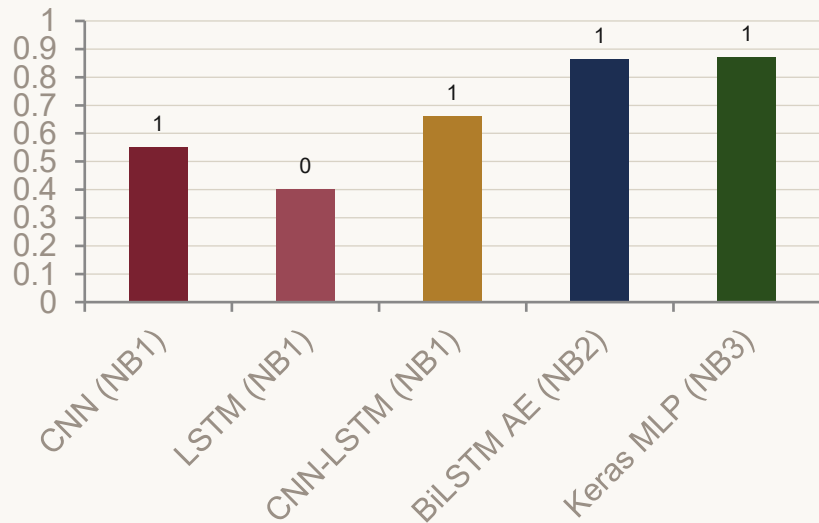
Per event — event detected if any sample is flagged. "Did we catch this failure?"

AUC-ROC = 0.5: no better than random
 AUC-ROC = 1.0: perfect separation
 AUC-ROC < 0.5: worse than random (e.g. LSTM)

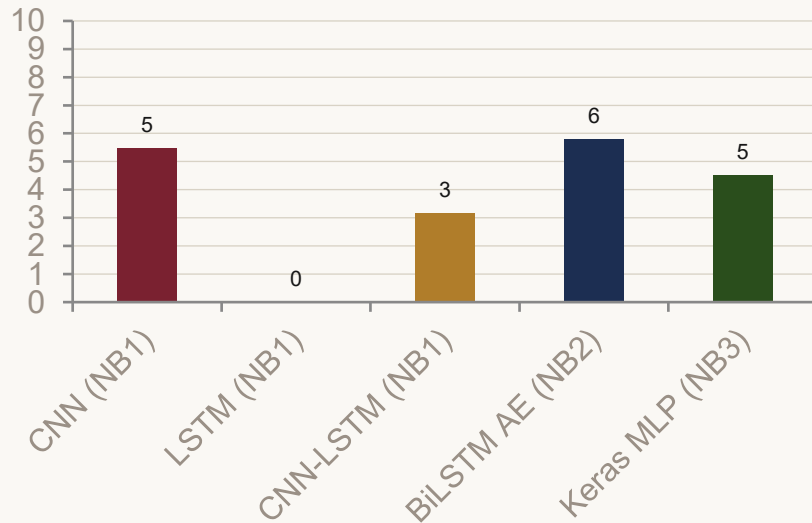
AUC-ROC measures whether the model ranks anomalies above normal samples regardless of threshold — the most honest metric for imbalanced data.

Sample-Level Performance

AUC-ROC by Model



F1-Score (%) by Model



0.55

CNN - AUC-ROC

0.40

LSTM - AUC-ROC (worse than random)

0.66

CNN-LSTM - AUC-ROC

0.865

BiLSTM AE - AUC-ROC

0.872

Keras MLP AE - AUC-ROC

Full Results — Sample-Level and Event-Level

SAMPLE-LEVEL METRICS (test set)

Model	Precision	Recall	F1	AUC-ROC
NB1 CNN	19.62%	3.18%	5.47%	0.55
NB1 LSTM	0.00%	0.00%	0.00%	0.40
NB1 CNN-LSTM	3.28%	3.05%	3.16%	0.66
NB2 P99	2.99%	88.46%	5.78%	0.865
NB3 Keras MLP	2.32%	88.26%	4.52%	0.872

Precision of ~2-3% across all models: the dominant challenge remains excessive false alarms from normal operational variability.

NB2 — EVENT DETECTION (P95 threshold)

Event	Anomalous Windows	Detected	Rate
Event 16	2,195	2,109	96.1%
Event 30	3,298	3,153	95.6%
Event 66	978	978	100.0%
Weighted avg.	-	-	96.4%

NB3 — CARE SCORE (58 events)

0.699

CARE Score

94.1%

Recall (events) 48 of 51 detected

88.9%

Precision (events) 48 of 54 alarms correct

Event-level evaluation is operationally superior: a fault event spans thousands of timesteps, so detecting even one correctly flags the whole event. This is what matters for maintenance decisions.

Summary and Model Ranking

1

NB3 — Keras MLP Autoencoder

AUC-ROC 0.872 · CARE 0.699 · Event Recall 94.1% · Precision 88.9%

2

NB2 — CNN-BiLSTM Autoencoder

AUC-ROC 0.865 · Event Recall 96.4% across 3 test events

3

NB1 — CNN-LSTM (best supervised)

AUC-ROC 0.66 · F1 3.16% · LSTM alone: 0% across all metrics

V

Semi-supervised outperforms supervised under severe imbalance — AUC-ROC 0.87 vs 0.66 maximum

X

Isolated LSTM fails completely when train/test balance differs

!

Low Precision (~2-3%) is the main open problem — too many false alarms

V

Event-level evaluation is more operationally meaningful than sample-level

Next Steps

Fix the high amount of False positives

Induce failure in the dataset using noise:

Classifier: train the model labelling this new Induced Anomaly

Autoencoder: train the model with Normal data and check the reconstruction error for detecting the Induced Anomalies



Thank you

Wind Turbine Anomaly Detection IC 2026



DATASET

CARE_To_Compare Wind Farm C

BEST RESULT

AUC-ROC 0.872 Event Recall 94.1%

AUTHOR

Thiago Heleno April 2026